Plant Disease Detection using Deep Learning
Instructor: Mr. Indraneel Pole & Mr. Andreas Pech
Student Name: Nguyen Do Nguyen - Date: 28/01/2026

1. Methodology

1.1 Dataset

The model is trained on a dataset provided by PlantVillage with a focus on Tomato leaf diseases. The dataset has 10 different classes, which are Healthy, Bacterial Spot, Early Blight, Late Blight, Leaf Mold, Septoria Leaf Spot, Spider Mites, Target Spot, Mosaic Virus, and Yellow Leaf Curl Virus. The images are resized to 256x256 pixels and normalized in the pixel range of [0, 1]. Then I will split into data into 3 group: Train (74%), Valid (14%), and Test (2%).

1.2 Data Preprocessing & Augmentation

To prevent overfitting and improve generalization, I use the tf.data pipeline after handling with these techniques:

First I rescale to 1./255 normalization. Then for the data augmentation, I try rotating images randomly (20%), randomly zooming (10%), and doing some horizontal/vertical flips. Finally, I apply .cache() a and .prefetch() to streamline data loading during training to optimize performance.

1.3 Model Architecture (Custom CNN)

I implement a Convolutional Neural Network (CNN) using TensorFlow and Keras. Here is the architecture:

- 5 Convolutional Blocks: Each block contains a Conv2D layer (for extracting), BatchNormalization (for training stablely), and MaxPooling2D (for reducing dimension).
- Global Average Pooling: Used instead of flattening to reduce the number of parameters and minimize overfitting.
- Dropout: A dropout rate of 0.5 was applied to randomly disable neurons during training.
- Output Layer: A Dense layer with Softmax activation to output probabilities for the 10 disease classes.

2. System Design (UML Diagrams)

2.1 Class Diagram

Diagram (1) shows the logical structure of the Python application that separates the User Interface (Streamlit) from the backend Logic (Model).

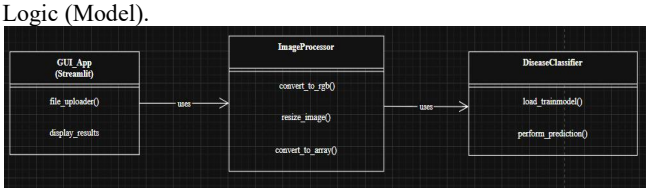

Diagram (1): Logical Structure

2.2 Activity Diagram (Process Flow)

This ***Diagram (2)*** describes the step-by-step flow when a user interacts with the system.

3. Results

3.1 Performance Metrics

The model is trained for 20 epochs using the Adam optimizer with a dynamic learning rate scheduler.

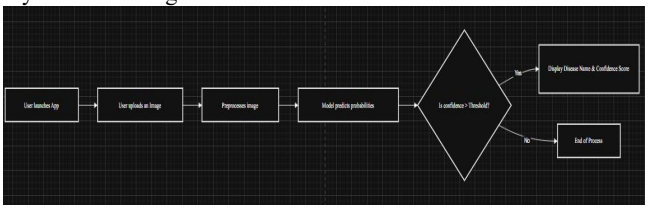

Diagram (2): Model of Adam optimizer

Final Training Accuracy: ~98%

Final Validation Accuracy: ~89.65%

Loss: The validation loss ***Figure (3)*** fluctuates around 0.3 which indicates that the model has successfully learned to generalize without a bad overfitting.

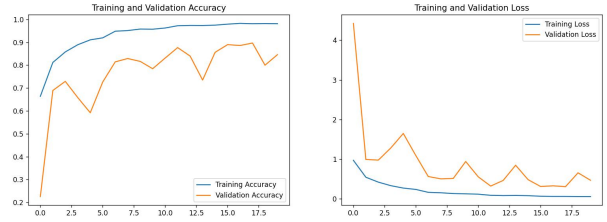

Figure (3): Traning and Validation Accuracy and Loss

Figure (4) shows the per-class performance. Diagonal elements indicates correct predictions. The matrix highlights that there are visual similarities between "Early Blight" and "Late Blight" caused minor misclassifications.

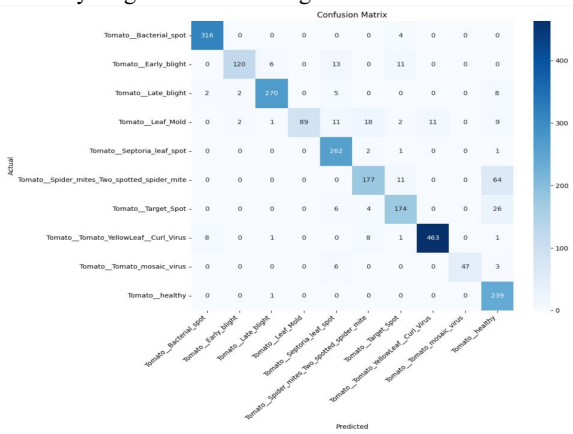

Figure (4): Confusion Matrix

Figure (5) recorded a high accuracy of 89.65% in validation, performing exceptionally well in different diseases such as Mosaic Virus and Bacterial Spot. The misclassification of a few instances in the Leaf Mold class indicates a need for some more improvement, still, it remains a highly reliable system.

	precision	recall	f1-score	support
Tomato__Bacterial_spot	0.9693	0.9875	0.9783	320
Tomato__Early_blight	0.9677	0.8000	0.8759	150
Tomato__Late_blight	0.9677	0.9408	0.9541	287
Tomato__Leaf_Mold	1.0000	0.6224	0.7672	143
Tomato__Septoria_leaf_spot	0.8647	0.9850	0.9209	266
Tomato__Spider_mites_Two_spotted_spider_mite	0.8469	0.7024	0.7679	252
Tomato__Target_Spot	0.8529	0.8286	0.8406	210
Tomato__Tomato_YellowLeaf__Curl_Virus	0.9768	0.9606	0.9686	482
Tomato__Tomato_mosaic_virus	1.0000	0.8393	0.9126	56
Tomato__healthy	0.6809	0.9958	0.8088	240
accuracy			0.8965	2406
macro avg	0.9127	0.8662	0.8795	2406
weighted avg	0.9098	0.8965	0.8957	2406

Figure (5): Classification Report

4. Conclusion

This project has successfully created a Deep Learning system that is capable of detecting Tomato Plant Diseases with an accuracy of more than 89%. The implementation of a Custom CNN model with data augmentation and a learning rate scheduler has effectively overcome common issues such as overfitting and data inconsistency.

The project has shown the real-world application of AI and Deep Learning in the field of agriculture. The system has the potential to save crops by allowing farmers to quickly identify problems with their crops using just a photograph. From there, it could potential to be developed further by integrating it into a mobile application or using Transfer Learning to achieve an accuracy of more than 95%.

Dataset used: https://www.kaggle.com/code/emmarex/plant-disease-detection-using-keras/input